SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I **M Sivam Kumar** *(192521017) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

M Sivam Kumar
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1. Errors in Equivalence Class Partitioning and Boundary Value Analysis

Error Identification

In the online shopping system, the tester incorrectly combines valid and invalid inputs into the same equivalence class and ignores boundary values. This makes the test design incomplete because inputs with different expected outcomes are tested as if they belong to the same category.

For example, consider a quantity field that accepts values from 1 to 100. Valid quantities are between 1 and 100, while zero, negative numbers, and values above 100 are invalid.

Root Causes

- Incorrect partitioning: Inputs with different expected results are placed in one class.
- Missing boundary analysis: Errors commonly occur at the minimum and maximum permitted values.
- Insufficient test selection: Testing only ordinary values may miss off-by-one errors and invalid-input handling.
- Weak expected-result definition: Each class is not clearly associated with its expected system response.

Corrected Test Cases

Equivalence class	Example input	Expected result
Invalid: below minimum	0	Reject
Valid: minimum quantity	1	Accept
Valid: ordinary quantity	50	Accept
Valid: maximum quantity	100	Accept
Invalid: above	101	Reject

maximum		
Invalid: negative quantity	-5	Reject

Boundary Value Analysis should test values around the limits: 0, 1, 2, 99, 100, and 101.

Justification

Equivalence Class Partitioning reduces redundant testing by selecting representative values from distinct input groups. Boundary Value Analysis complements it by examining values at and around the limits. Using both techniques improves defect detection, especially for incorrect comparisons, validation mistakes, and off-by-one errors. It also gives a more systematic and reliable test suite than selecting inputs randomly.

2. Errors in Random Testing of a Banking Application

Error Identification

Random testing alone is not a sufficient strategy for a banking application. Random inputs may exercise common operations while failing to test important transaction limits, invalid amounts, account restrictions, and boundary conditions.

The engineer also incorrectly assumes that high test execution or input volume automatically means high test coverage. Test volume and test adequacy are not the same thing.

Root Causes

1. Random values may rarely fall exactly on critical boundaries.
2. Test cases may not cover every valid and invalid transaction category.
3. Important combinations, such as insufficient balance and maximum withdrawal limits, may be overlooked.
4. No systematic coverage criteria are defined.
5. Expected results and business rules may not be checked for every transaction.

Corrected Testing Strategy

Suppose a banking application allows withdrawals from ₹100 to ₹50,000, subject to the available account balance and other bank rules.

Testing technique	Example test	Expected result
Equivalence Class Partitioning	Withdraw ₹5,000 with sufficient funds	Accept

Equivalence Class Partitioning	Withdraw a negative amount	Reject
Boundary Value Analysis	Withdraw ₹99, ₹100, ₹101	Reject, accept, accept
Boundary Value Analysis	Withdraw ₹49,999, ₹50,000, ₹50,001	Accept, accept, reject, subject to other limits
Decision Table Testing	Valid account, sufficient funds, within limit	Approve
Decision Table Testing	Valid account but insufficient funds	Reject
State Transition Testing	Pending transaction followed by success or failure	Correct status and balance update

The exact limits should be taken from the application's documented business rules.

Justification

Structured black-box techniques ensure that inputs, boundaries, and combinations are tested deliberately. Random testing can still be used as an additional technique for discovering unexpected defects, but it should not replace systematic test design. Combining these methods improves coverage, reduces missed critical cases, and increases confidence in transaction reliability.

3. Errors in Control Flow Graph Construction and Path Testing

Error Identification

A Control Flow Graph (CFG) represents the possible execution flow of a program using nodes for statements or blocks and directed edges for control transfers. If a developer omits decision nodes or loops, the graph no longer represents the actual program accurately.

As a result, some execution paths may not be tested, and defects in conditional logic or repeated execution may remain undetected.

Root Causes

- Decision conditions are treated as ordinary statements rather than branching points.

- True and false outcomes are not represented separately.
- Loop entry, repetition, and exit edges are missing.
- Independent paths are not identified before selecting test cases.
- Coverage is judged from the incomplete graph rather than the actual source code.

Corrected Control Flow Representation

Consider the following illustrative code:

```
1. Start
2. Read amount
3. If amount > 0
4.     If balance >= amount
5.         Deduct amount
6.     Else
7.         Display "Insufficient balance"
8. Else
9.     Display "Invalid amount"
10. End
```

The corrected CFG should contain:

- An entry node and an input node.
- A decision node for `amount > 0`.
- A separate decision node for `balance >= amount`.
- A processing node for deducting the amount.
- Separate error-handling nodes for invalid amount and insufficient balance.
- A common exit node.

Independent Paths and Test Cases

Path	Execution flow	Example input	Expected result
P1	Start → input → amount check (false) → invalid amount → End	Amount = -10	Invalid amount message
P2	Start → input → amount check (true) → balance check (true) → deduction → End	Amount = 100, balance = 500	Balance reduced by 100
P3	Start → input → amount check (true) → balance check (false) → insufficient balance → End	Amount = 600, balance = 500	Transaction rejected

These paths are independent in the sense that each exercises a distinct outcome of the decisions. The exact number of independent paths for a real program must be derived from its

complete CFG.

Justification

A complete CFG makes all decision outcomes, loop paths, and exits visible. Basis path testing can then identify a set of independent paths that should be executed at least once. This improves branch and path coverage and helps detect logical errors, missing error handling, and incorrect control transfers. For loops, additional tests must cover zero, one, and multiple iterations as appropriate.

4. Why 100% Statement Coverage Is Not Enough

Error Identification

Statement coverage measures whether each executable statement has run at least once. It does not guarantee that every decision outcome has been tested.

A test suite can execute all statements while still failing to detect incorrect conditions, missing branches, or logical errors. Therefore, claiming complete testing based only on 100% statement coverage is an inadequate conclusion.

Root Causes

1. Only statement execution is measured.
2. Both true and false outcomes of decisions are not necessarily exercised.
3. Compound conditions may not be tested with different combinations of Boolean values.
4. Error-handling branches may remain untested.
5. Coverage percentage is mistaken for proof that the software is defect-free.

Example

```
if (age >= 18)
   display "Eligible"
else
   display "Not eligible"
```

If the only test input is `age = 20`, the statement `display "Eligible"` executes, but the `else` statement does not. Statement coverage is therefore not 100% for this two-branch example.

A stronger example is:

```
if (age >= 18)
   eligible = true
else
   eligible = false
display eligible
```

Testing `age = 20` and `age = 16` executes every statement, giving 100% statement coverage. However, further tests around the threshold are still useful to detect boundary errors.

Improved Testing Strategy

- Statement coverage: Execute every executable statement.
- Branch coverage: Execute both true and false outcomes of each decision.
- Condition coverage: Make each individual Boolean condition evaluate to both true and false.
- Path testing: Test important independent execution paths, subject to practical limits where loops create many paths.
- Boundary Value Analysis: Test `age = 17`, `18`, and `19` to verify the eligibility threshold.

Justification

Branch coverage detects errors that statement coverage may miss because it requires every decision outcome to be exercised. Condition coverage provides additional assurance for compound expressions, while path testing explores combinations of control flow. Using suitable coverage criteria together creates a more effective test suite than relying on statement coverage alone.

5. Incorrect Equivalence Classes for an Age-Validation Field

Error Identification

The tester defines only two classes—negative numbers and positive numbers—for an age field that accepts values from 0 to 100. This partitioning is incorrect because it does not distinguish valid ages from invalid positive values greater than 100.

It also overlooks special values such as zero, values beyond the upper limit, and non-numeric inputs.

Root Causes

- Valid and invalid inputs are not divided according to the actual requirements.
- The upper limit of 100 is ignored.
- Boundary Value Analysis is not performed.
- Input data types are not considered.
- Representative test cases are not selected from every meaningful class.

Corrected Equivalence Classes

Class	Input example	Expected result
Invalid: negative age	-1	Reject
Valid: minimum age	0	Accept
Valid: ordinary age	35	Accept

Valid: maximum age	100	Accept
Invalid: age above maximum	101	Reject
Invalid: non-numeric input	`"abc"`	Reject
Invalid: decimal age, if only whole years are allowed	25.5	Reject

The decimal-input rule depends on whether the system requires integer ages.

Boundary Value Analysis

Test the values immediately around both boundaries:

- Lower boundary: `-1, 0, 1`
- Upper boundary: `99, 100, 101`

Justification

Correct equivalence classes ensure that inputs with the same expected behavior are grouped together. Boundary Value Analysis focuses on the points most likely to expose comparison and validation errors. Together, they improve robustness by checking valid limits, out-of-range values, and incorrect data types. This is more effective than dividing inputs merely into negative and positive numbers.

6. Errors in Condition Coverage for a Complex Boolean Expression

Note: The supplied question is incomplete and ends at `if(A && B`. The following answer assumes the question concerns a developer who claims full condition coverage for the expression `A && B`.

Error Identification

For the Boolean expression `A && B`, the overall decision is true only when both conditions A and B are true. A common testing error is to test only the overall decision result and then claim full condition coverage.

Condition coverage requires each individual condition to evaluate to both true and false at least once. In some programming languages, short-circuit evaluation also means B may not be evaluated when A is false.

Root Causes

- Decision coverage and condition coverage are treated as identical.
- The test suite does not independently vary A and B.
- Short-circuit evaluation is overlooked.
- Compound Boolean expressions are not analysed condition by condition.

Corrected Test Cases

Test case	A	B	Expected decision
T1	True	True	True
T2	True	False	False
T3	False	True	False
T4	False	False	False

These four combinations demonstrate the behavior of the expression and ensure each condition is true and false across the test set. In a short-circuit language, B may not actually be evaluated in T3 or T4; that detail should be considered when assessing instrumentation-based condition coverage.

Justification

Condition coverage checks the individual Boolean components instead of only the final decision. Testing different combinations helps reveal incorrect operators, missing conditions, and logic mistakes. For safety- or business-critical expressions, modified condition/decision coverage (MC/DC) may provide stronger evidence by demonstrating that each condition can independently affect the overall decision outcome.

7. Errors in Decision Table Testing for a Discount System

Error Identification

The QA team has missed combinations of Premium Member and Coupon Applied, resulting in incorrect price calculations. The main error is an incomplete decision table that does not represent all meaningful combinations of the input conditions.

Decision Table Testing is useful when the output depends on multiple business rules. Missing a combination can cause a discount to be applied incorrectly, omitted, or applied twice.

Root Causes

1. Some combinations of membership status and coupon status are missing.
2. The expected discount for each combination is not clearly defined.
3. Conflicting rules may exist for combining membership and coupon discounts.
4. The tester may have assumed that testing one discount condition is sufficient to validate the entire calculation.
5. No systematic review has been performed to ensure all relevant rules are covered.

Corrected Decision Table

Assume that premium members receive a 10% discount, a valid coupon provides a 5% discount, and both discounts can be combined. These are illustrative rules; the actual table must follow the application's requirements.

Rule	Premium member?	Coupon applied?	Expected action
R1	Yes	Yes	Apply both permitted discounts
R2	Yes	No	Apply 10% premium discount
R3	No	Yes	Apply 5% coupon discount
R4	No	No	Apply no discount

If the business rules prohibit combining discounts, R1 must instead apply the specified priority rule, such as choosing only the higher discount. The requirement must explicitly define this behavior.

Test Cases

- Premium member with a valid coupon.
- Premium member without a coupon.
- Non-premium member with a valid coupon.
- Non-premium member without a coupon.
- Invalid or expired coupon, if supported by the system.
- Coupon applied to an ineligible product, if product restrictions exist.

Justification

A decision table systematically maps condition combinations to expected actions. It reduces the possibility of missing important business rules and makes the test design easy to review. By testing every meaningful combination, the QA team can identify incorrect discount

calculations, conflicting rules, and missing validations before release.

8. Errors in Loop Testing Strategy

Error Identification

The tester checks only the first and maximum iterations of a loop, such as `n = 1` and `n = max`. This is insufficient because loop defects can occur when the loop executes zero times, when it executes multiple times, or when input exceeds the permitted maximum.

Testing only the first and last iterations does not establish that the loop's entry condition, termination condition, and repeated execution are correct.

Root Causes

- Zero-iteration behavior is ignored.
- The case `n = max + 1` is not tested.
- Off-by-one errors in loop conditions are overlooked.
- The tester does not check how variables change between iterations.
- Nested loops and interactions between loop counters are not adequately tested.

Corrected Loop Testing Approach

For a simple loop with an allowed iteration count from 1 to `max`, use the following tests:

Test case	Input	Purpose
T1	`n = 0`	Check zero-iteration behavior
T2	`n = 1`	Check minimum permitted iteration
T3	`n = 2`	Check repeated execution
T4	`n = max - 1`	Check near-maximum execution
T5	`n = max`	Check maximum permitted iteration

T6	`n = max + 1`	Check handling of an excessive value

If zero is invalid, the system should reject it or handle it according to the requirements. If zero iterations are permitted, the loop should terminate safely without executing its body.

Testing Nested Loops

For nested loops, a suitable strategy is:

1. Test the inner loop independently while keeping the outer loop at a simple value.
2. Test the outer loop with the inner loop executing zero, one, and multiple times, where permitted.
3. Test combinations near both loops' limits.
4. Check termination and ensure the counters are updated correctly.
5. Test abnormal inputs to confirm that the program does not enter an infinite loop.

Exhaustive testing of every combination may be impractical for large nested loops, so priority should be given to important boundary combinations and independent loop behavior.

Justification

A systematic loop-testing strategy detects off-by-one errors, incorrect termination conditions, unexpected zero-iteration behavior, and repeated-execution defects. Testing nested loops at different iteration counts also helps verify that one loop does not interfere incorrectly with another. This improves control-flow reliability and reduces the risk of infinite loops or incorrect results.

9. Errors in Using Error Guessing Alone for a Safety-Critical Braking System

Error Identification

Error Guessing uses the tester's experience and intuition to anticipate likely defects. It can be useful, but relying on it as the primary or sole technique for a safety-critical braking system is inadequate.

A braking system must respond correctly to different speeds, sensor readings, braking requests, and operating states. Informal testing cannot guarantee systematic coverage of critical limits and state transitions.

Root Causes

- Test cases depend heavily on individual experience.
- Important input boundaries may be overlooked.
- Different operating states and transitions may not be covered.
- There may be no traceable link between safety requirements and test cases.
- The testing process lacks systematic evidence that critical behavior has been verified.

Corrected Strategy: Combine BVA and State Transition Testing

1. Boundary Value Analysis (BVA)

Suppose, purely for illustration, a braking controller accepts a speed input from 0 to 200 km/h. Test values around the boundaries:

Input speed	Testing purpose
-1 km/h	Check invalid lower input
0 km/h	Check minimum boundary
1 km/h	Check just above minimum
199 km/h	Check just below maximum
200 km/h	Check maximum boundary
201 km/h	Check invalid upper input

The actual limits and expected braking behavior must come from the system's approved safety requirements. Physical braking response cannot be validated solely through numerical input checks.

2. State Transition Testing

Model important states such as:

- Normal operation
- Braking requested
- Braking active
- Vehicle stopped
- Sensor fault or system fault

Test valid and invalid transitions, including:

- Normal operation → braking requested → braking active.
- Braking active → vehicle stopped.

- Normal operation → sensor fault → safe fault response.
- Recovery from a fault, where the requirements permit it.

The system must also be tested for delayed, repeated, or contradictory sensor signals where relevant.

3. Retain Error Guessing as a Supplement

Use tester experience to suggest additional cases, such as sensor disconnection, unexpected input values, or abrupt changes in sensor readings. These cases should supplement—not replace—requirements-based and structured testing.

Justification

BVA targets defects near important input limits, while State Transition Testing verifies that the system behaves correctly as it moves between operating states. Combining them provides more systematic coverage than error guessing alone. For a safety-critical braking system, these techniques must also be supported by risk analysis, requirements traceability, fault-injection testing where appropriate, and the applicable safety verification process.

10. Errors in a Control Flow Graph with Missing `else` Branches

Error Identification

The tester has omitted the `else` branches of multiple nested `if` statements from the Control Flow Graph. This makes the graph incomplete because it represents only some possible execution paths.

As a result, a test suite may execute all statements on the successful paths while never exercising the error-handling branches. Statement coverage alone can therefore give a misleading impression of test adequacy.

Root Causes

1. False outcomes of decisions are not represented.
2. Nested decisions are simplified incorrectly.
3. Error-handling statements are omitted from the graph.
4. Independent paths are not identified correctly.
5. Coverage is measured using the incomplete CFG rather than the actual code.

Corrected Control Flow Graph

Consider this illustrative program:

```
Start
  |
Check input validity
  | True
  v
Check account status
  | True
  v
```

Process transaction
|
End

Check input validity -- False --> Display invalid input --> End

Check account status -- False --> Display account error --> End

The corrected CFG must include both outcomes of each decision:

- The true branch of the input-validation decision.
- The false branch leading to the invalid-input message.
- The true branch of the account-status decision.
- The false branch leading to the account-error message.
- The transaction-processing node and the relevant exit paths.

The graph must also represent any additional nested decisions and their corresponding `else` branches that appear in the actual source code.

Independent Paths and Test Cases

Path	Execution flow	Example test	Expected result
P1	Start → input invalid → error message → End	Malformed input	Invalid-input message
P2	Start → input valid → account active → process transaction → End	Valid input and active account	Transaction processed
P3	Start → input valid → account inactive → account error → End	Valid input and inactive account	Transaction rejected

These paths cover the distinct outcomes shown in the illustrative code. A real module may require additional paths for further nested decisions, loops, or error conditions.

Improved Testing Approach

- Construct the CFG directly from the complete source code.
- Represent each decision and its true and false outcomes.
- Include all error-handling nodes and exit paths.
- Identify independent paths using the complete graph.
- Create test cases that exercise each branch.
- Measure branch coverage in addition to statement coverage.

- Add condition coverage for complex Boolean expressions.

Justification

A complete CFG reveals the program's actual control structure and makes missing branches easier to identify. Branch coverage ensures that both true and false outcomes are exercised, while path testing checks important combinations of decisions. This is particularly valuable for nested conditions because error-handling code may otherwise remain untested. The corrected approach improves defect detection and provides stronger evidence that the module handles both normal and abnormal situations correctly.

Conclusion

Effective software testing requires more than executing a large number of test cases. Testers must identify input classes, examine boundary values, represent control flow accurately, and verify decision outcomes and important execution paths.

Equivalence Class Partitioning and Boundary Value Analysis improve input validation, while Decision Table and State Transition Testing address combinations of rules and changing system states. White-box techniques such as branch coverage, condition coverage, and path testing help reveal defects hidden by incomplete execution coverage.

By correcting the identified errors and applying structured testing techniques alongside experience-based methods, the testing process becomes more systematic, measurable, and reliable. These improvements increase the likelihood of detecting defects before deployment and support better software quality and safety.